Universidad de Ingeniería

Trabajo de Fin de Grado

WinDebSc: Herramienta de Automatización y Optimización de Windows en Java

Autor: Alealfras

Grado en Ingeniería Informática

Fecha: 10 de noviembre de 2026

Resumen

Este documento describe el desarrollo de **WinDebSc**, una aplicación de escritorio desarrollada en Java diseñada para centralizar, automatizar y gestionar el proceso de optimización y eliminación de bloatware (debloating) en sistemas operativos Windows. Inspirada en utilidades como la herramienta de Chris Titus, esta solución integra ejecución segura de scripts de PowerShell, personalización de directivas de sistema y una interfaz intuitiva con trazabilidad de acciones y restauración ante fallos.

Índice general

Capítulo 1

Introducción y Motivación

1.1. Contexto del Problema

Los sistemas operativos modernos, en especial Windows 10 y Windows 11, incluyen por defecto una gran cantidad de servicios en segundo plano, recolección de telemetría y software preinstalado comúnmente denominado *bloatware*. Estos componentes consumen recursos del hardware (CPU, memoria RAM, lecturas/escrituras en disco) y plantean consideraciones relevantes sobre la privacidad del usuario.

1.2. Inspiración y Objetivos

Existen soluciones en la comunidad de código abierto, destacando la popular herramienta *WinUtil* de Chris Titus. El objetivo de este proyecto es diseñar una suite propia con interfaz gráfica desarrollada en **Java**, capaz de orquestar scripts de automatización modulares y reproducibles en el sistema anfitrión.

1.3. Estructura de la Memoria

La presente memoria documenta todo el ciclo de vida del software: análisis de requerimientos, diseño modular, implementación en Java y PowerShell, y batería de pruebas.

Capítulo 2

Fundamentos Teóricos y Estado del Arte

2.1. Concepto de Debloating y Telemetría

El término *debloating* hace referencia al proceso sistemático de remover o deshabilitar aplicaciones preinstaladas, servicios redundantes y mecanismos de telemetría en un sistema operativo. En el ecosistema Windows 10 y Windows 11, la telemetría recopila eventos de diagnóstico del sistema mediante componentes como el servicio de Experiencias del Usuario y Telemetría Asociadas (`DiagTrack`). Si bien estos datos permiten a los desarrolladores identificar errores, introducen sobrecarga continua en ciclos de CPU, I/O en disco y debates en torno a la privacidad de los datos personales.

2.2. Seguridad y Políticas de Ejecución de PowerShell

PowerShell implementa directivas de ejecución (*Execution Policies*) diseñadas no como barreras de seguridad infranqueables, sino como medidas preventivas frente a la ejecución accidental de scripts maliciosos. Por defecto, los clientes Windows restringen la ejecución de scripts no firmados (`Restricted`).

Para permitir la automatización sin vulnerar de forma permanente la configuración global del registro de Windows, se emplea el parámetro `-ExecutionPolicy Bypass` acotado exclusivamente al proceso hijo generado. Esto garantiza que la directiva permisiva solo viva durante el ciclo de vida del proceso invocado, restableciendo la seguridad del host inmediatamente tras su finalización.

2.3. Interoperabilidad entre Java (ProcessBuilder) y el Sistema Operativo

La plataforma Java proporciona la API `java.lang.ProcessBuilder` para instanciar subprocesos a nivel de sistema operativo. A diferencia de las llamadas primitivas heredadas

de `Runtime.getRuntime().exec()`, `ProcessBuilder` ofrece:

- Control explícito de la tabla de variables de entorno.
- Redirección y combinación de los descriptores estándar de entrada, salida y error (`stdin`, `stdout`, `stderr`).
- Sincronización precisa mediante el código de salida (*exit code*), respetando el estándar POSIX donde el valor 0 denota terminación correcta y valores distintos de cero indican condiciones de error.

Capítulo 3

Análisis de Requisitos

3.1. Requisitos Funcionales (RF)

- **RF-1 (Seguridad Preventiva)**: El sistema debe permitir la creación de un punto de restauración del sistema (*System Restore Point*) antes de la aplicación de cambios destructivos o modificaciones del registro.
- **RF-2 (Verificación de Privilegios)**: El sistema debe validar la presencia de privilegios administrativos (UAC) antes de despachar tareas que modifiquen servicios o directivas del sistema.
- **RF-3 (Ejecución Asíncrona y Streaming)**: La interfaz debe consumir y mostrar las trazas de ejecución de PowerShell en tiempo real sin congelar el hilo principal.

3.2. Requisitos No Funcionales (RNF)

- **RNF-1 (Modularidad)**: Los scripts de automatización deben ser independientes y desacoplados del código Java.
- **RNF-2 (No Intrusividad)**: La aplicación no debe modificar permanentemente las directivas globales de PowerShell en la máquina anfitriona.

Capítulo 4

Diseño e Implementación

4.1. Arquitectura del Sistema

El sistema sigue un modelo desacoplado en dos capas:

1. **Capa de Orquestación y Presentación (Java)**: Responsable de la lógica de negocio, verificación de prerequisitos, selección de módulos y presentación de trazas.
2. **Capa de Actuación (PowerShell)**: Scripts modulares con acceso nativo al Windows Management Instrumentation (WMI), registro del sistema y administración de servicios.

4.2. Módulo de Seguridad Preventiva (`restore-point.ps1`)

El script `restore-point.ps1` implementa la primera línea de defensa. Realiza las siguientes operaciones:

1. Valida que la identidad del subproceso pertenezca al rol nativo de Administrador mediante `WindowsBuiltInRole::Administrator`.
2. Activa la directiva de restauración en el volumen raíz (`C:\`) si estuviese previamente inactiva mediante `Enable-ComputerRestore`.
3. Genera una instantánea con el cmdlet `Checkpoint-Computer` bajo la categoría `MODIFY_SETTINGS`.

4.3. Gestor de Procesos en Java (`ScriptRunner.java`)

La clase `ScriptRunner` encapsula la comunicación interproceso. A continuación se ilustra la invocación parametrizada y la captura en tiempo real mediante un consumidor funcional:

```
1 ProcessBuilder processBuilder = new ProcessBuilder(
2     "powershell.exe",
```

```
    "-NoProfile",
    "-ExecutionPolicy", "Bypass",
    "-File", scriptFile.getAbsolutePath()
);
processBuilder.redirectErrorStream(true);
Process process = processBuilder.start();

try (BufferedReader reader = new BufferedReader(
        new InputStreamReader(process.getInputStream(), StandardCharsets
    .UTF_8))) {
    String line;
    while ((line = reader.readLine()) != null) {
        outputListener.accept(line);
    }
}
return process.waitFor();
```

Listing 4.1: Ejecución controlada de PowerShell con ProcessBuilder

Capítulo 5

Pruebas y Validación

5.1. Pruebas en Entorno Controlado (Máquina Virtual)

Capítulo 6

Conclusiones y Trabajo Futuro